

Optimization for Data Science

HOMEWORK

Nicolò Borga, Matteo Randa, Andrea Senatore

1 Introduction

This report focuses on the role of optimization in addressing machine learning problems. The goal is to minimize a loss function over model parameters. In this work, the focus is on a multiclass logistic regression problem, where the objective function is the negative log-likelihood, which is convex. The problem structure depends on three key parameters: the number of samples m , the dimension d and the number of classes k . When these parameters are large, the problem becomes computationally challenging. The algorithms that will be implemented are: gradient descent and block coordinate gradient descent with randomized rule and with Gauss-Southwell rule.

The main focus of this report is the comparison between these three methods. Gradient descent updates all variables at each iteration, making it computationally expensive, while BCGD methods update only part of the variables at each iteration, leading to a cheaper iterations but potentially slower convergence. The methods are evaluated in terms of accuracy and CPU time. Two analyses are conducted: one on synthetic data and one on a real world dataset.

2 Formulation

In order to clearly describe the problem formulation, it is crucial to define the input of the model. The input is represented by a matrix $A \in \mathbb{R}^{m \times d}$, where each row corresponds to a sample described by a feature vector $a_i \in \mathbb{R}^d$ and an associated label $b_i \in \{1, \dots, k\}$. It is also necessary to introduce the model parameters, which represent the optimization variable. These are denoted by $X \in \mathbb{R}^{d \times k}$, where each column $x_c \in \mathbb{R}^d$ corresponds to the weight vector associated with class c .

Having defined all the main components, the optimization problem can be now formulated. As mentioned before, it is a multiclass logistic regression problem of the form:

$$\min_{X \in \mathbb{R}^{d \times k}} \sum_{i=1}^m \left(-x_{b_i}^\top a_i + \log \left(\sum_{c=1}^k e^{x_c^\top a_i} \right) \right) \quad (1)$$

The objective function can be interpreted as the sum of two terms. The first term, $-x_{b_i}^\top a_i$, represents the score assigned by the model to the correct class. The second term, $\log \left(\sum_{c=1}^k e^{x_c^\top a_i} \right)$, acts as a normalization factor and ensures a probabilistic interpretation.

In order to interpret the model probabilistically, the softmax function is introduced.

$$P(b_i | a_i, X) = \frac{e^{x_{b_i}^\top a_i}}{\sum_{c'=1}^k e^{x_{c'}^\top a_i}} \quad (2)$$

It is a probability distribution on the classes. Normalizes the logits defined as $x_{b_i}^\top a_i$. To maximize the likelihood over m independent identically distributed training samples, the negative likelihood (which is the objective function) can be minimized.

In order to apply all the algorithms analyzed, the derivative must be shown. First we show the partial derivative of the objective with regards to a generic x_{ic} :

$$\frac{\partial f(x)}{\partial x_{jc}} = - \sum_{i=1}^m a_{ij} \left[\mathbb{I}(b_i = c) - \frac{e^{x_c^\top a_i}}{\sum_{c'=1}^k e^{x_{c'}^\top a_i}} \right] \quad (3)$$

where $\mathbb{I}$ is the indicator function for the example i :

$$\mathbb{I}(b_i = c) = \begin{cases} 1 & b_i = c, \\ 0 & \text{otherwise} \end{cases}$$

The matrix formulation is the following:

$$A^T (P - \mathbb{I}) \quad (4)$$

where $\mathbb{I}$ is the indicator function explained before, but for all examples.

The objective function is convex, this can be proved in 2 ways. The first one is by noticing that the objective is the a combination of 2 elements: $-x_{b_i}^\top a_i$ that it's convex itself and $\log \left(\sum_{c=1}^k e^{x_c^\top a_i} \right)$ that it's also convex. This can be explained by the fact that $x_c^\top a_i$ is convex, the logarithm and the exponential are convex functions, so the function of convex function is convex itself. By considering this, the sum of 2 convex function is convex too. This prove the convexity.

The second way is the mathematical proof. The proof exploits the semi-definite positive property of the Hessian, this condition is sufficient to prove convexity (it will be explained

better after the proof of convexity). Let's consider the function for a single example, and split it in 2. The first part is $x_{b_i}^T a_i$ that is linear with regard to x_{b_i} , this means that the Hessian is

$$H(x_{b_i}^T a_i) = \frac{\partial}{\partial x_{b_i}} a_i = 0 \quad (5)$$

This means that $x_{b_i}^T a_i$ is convex. Now let's look at the second element of the function: $f(x) = \log \left(\sum_{c=1}^k e^{x_c^T a_i} \right)$. The derivative is for a general $j \in [0, k-1]$:

$$\nabla_{c_j} f(x) = \frac{\partial}{\partial x_{c_j}} \log \left(\sum_{c=1}^k e^{x_c^T a_i} \right) = \frac{1}{\sum_{c=1}^k e^{x_c^T a_i}} e^{x_{c_j}^T a_i} = P(c_j \mid a_i, x) a_i^T = p_{ic_j} a_i^T \quad (6)$$

Now, by exploiting the structure of the second derivative:

$$\nabla_{c_h} (\nabla_{c_j} f(x)) = \frac{\partial}{\partial x_{c_h}} p_{ic_j} a_i^T = \frac{-e^{x_{c_j}^T a_i} e^{x_{c_h}^T a_i}}{\left(\sum_{c=1}^k e^{x_c^T a_i} \right)^2} = -p_{ic_j} p_{ic_h} a_i^T, \quad \forall j \neq h \quad (7)$$

$$\frac{\partial^2}{\partial c_j^2} f = \frac{\partial}{\partial c_j} p_{ic_j} a_i^T = p_{ic_j} (1 - p_{ic_j}) a_i^T, \quad j = h \quad (8)$$

Therefore, the Hessian becomes the following matrix

$$H_{c_j, c_h} = \begin{cases} p_{ic_j} (1 - p_{ic_j}) a_i a_i^T & j = h, \\ -p_{ic_j} p_{ic_h} a_i a_i^T & j \neq h \end{cases} \quad (9)$$

To show that H is positive semi-definite, $v^T H v \geq 0$ for any arbitrary vector $v \in \mathbb{R}^{dk}$. The vector has these dimensions because the matrix H is composed of blocks of dimensions $d \times d$ each, where the block ij is the mixed second derivatives with regards to the classes i, j . This is given by the fact that we vectorize X from a matrix in $\mathbb{R}^{d \times k}$ to a vector in $\mathbb{R}^{dk}$ in order to compute the Hessian, otherwise we would have to operate in the tensor realm. Let's partition v into k blocks of length d : $v = [v_1^T, \dots, v_k^T]^T$. Evaluating the quadratic form:

$$v^T H v = \sum_{c=1}^k \sum_{c'=1}^k v_c^T H_{cc'} v_{c'} = \sum_{c=1, c'=c}^k v_c^T H_{cc} v_c + \sum_{c=1}^k \sum_{c'=1, c' \neq c}^k v_c^T H_{cc'} v_{c'} \quad (10)$$

$$\sum_{c=1}^k v_c^T (p_{ic}(1 - p_{ic}) a_i a_i^T) v_c + \sum_{c=1}^k \sum_{c' \neq c}^k v_c^T (-p_{ic} p_{ic'} a_i a_i^T) v_{c'} \quad (11)$$

Notice that $v_c^T a a^T v_{c'} = (v_c^T a)(a^T v_{c'}) = (v_c^T a)(v_{c'}^T a) = u_c u_{c'}$, where u_c is a scalar. Substituting this back in the quadratic form:

$$v^T H v = \sum_{c=1}^k (p_{ic}(1 - p_{ic})) u_c^2 - \sum_{c=1}^k \sum_{c' \neq c}^k p_{ic} p_{ic'} u_c u_{c'} = \sum_{c=1}^k p_c u_c^2 - \sum_{c=1}^k \sum_{c'=1}^k p_{ic} p_{ic'} u_c u_{c'} \quad (12)$$

$$= \sum_{c=1}^k p_{ic} u_c^2 - \left(\sum_{c=1}^k p_{ic} u_c \right) \left(\sum_{c'=1}^k p_{ic'} u_{c'} \right) = \sum_{c=1}^k p_{ic} u_c^2 - \left(\sum_{c=1}^k p_{ic} u_c \right)^2 \quad (13)$$

Because $\sum_{c=1}^k p_{ic} = 1, p_{ic} > 0$, we can interpret p_{ic} as a probability distribution and express the formula above as the variance of a discrete random variable U that takes value u_c with probability p_{ic} :

$$\mathbb{V}[U] = \mathbb{E}[U^2] - \mathbb{E}[U]^2 \quad (14)$$

Since variance is always non-negative, we conclude

$$v^T H v \geq 0 \quad (15)$$

Because H is semi-definite positive for all X , the function $f(X)$ is convex with respect to the matrix X . Together with the linear part, it can be concluded that the objective for a multi-class logistic function is convex.

As said before, the condition of semi-definite positive of the Hessian matrix is sufficient to prove convexity.

Another important element, coming from the theory, used in the algorithms that require a proof is the Lipschitz constant L , that is the fixed stepsize. In order to prove the value of L , the norm of the Hessian must be bounded by L :

$$\|H\|_2 \leq L \quad \forall X \quad (16)$$

The tightest bound is taken, that is $L = \lambda_{max}$ the largest eigenvalue of the Hessian. The Hessian has already been formulated in the previous proof, let's now consider the vector of probabilities p_i of example i , and the diagonal matrix $P_i = \text{diag}(p_i)$. The Hessian can be written as:

$$\frac{\partial f(X)}{\partial x_{jc} \partial x_{lc'}} = \sum_{i=1}^m a_{ij} a_{il} (p_{ic} \delta_{cc'} - p_{ic} p_{ic'}) = \sum_{i=1}^m (P_i - p_i p_i^T) \otimes (a_i a_i^T) \quad (17)$$

where $P_i - p_i p_i^T \in \mathbb{R}^{k \times k}$ and $\delta_{cc'}$ is the Kronecker delta that is 1 if $c = c'$ and 0 otherwise. $\otimes$ is the Kronecker product, that multiplies each element of the first matrix for all the second matrix, the result is a matrix of blocks. The matrix $H_i = P_i - p_i p_i^T$ is the covariance matrix of a multinomial distribution. For any vector $u \in \mathbb{R}^k$, the quadratic form is: $u^T H_i u = \sum_{c=1}^k p_{ic} u_c^2 - \left(\sum_{c=1}^k p_{ic} u_c \right)^2$. This is represent the variance as already said. The maximum eigenvalue is bounded by $\frac{1}{2}$. To find the maximum eigenvalue of $H = P - pp^T$, the maximum value of the quadratic form $u^T H u$, for any unit vector u ($\|u\|^2 = \sum u_c^2 = 1$), must be found. By the Popoviciu's Inequality:

$$\mathbb{V}(U) = u^T H u \leq \frac{1}{4} (u_{max} - u_{min})^2 \quad (18)$$

Now, we expand the squared difference:

$$(u_{max} - u_{min})^2 = u_{max}^2 + u_{min}^2 - 2u_{max}u_{min} \leq u_{max}^2 + u_{min}^2 + u_{max}^2 + u_{min}^2 \quad (19)$$

$$= 2(u_{max}^2 + u_{min}^2) \quad (20)$$

since

$$(u_{max} + u_{min})^2 = u_{max}^2 + u_{min}^2 + 2u_{max}u_{min} \geq 0 \implies -2u_{max}u_{min} \leq u_{max}^2 + u_{min}^2 \quad (21)$$

Now, knowing

$$u_{max}^2 + u_{min}^2 \leq \sum_{c=1}^k u_c^2 = \|u\|^2 \implies (u_{max} - u_{min})^2 \leq 2\|u\|^2 \quad (22)$$

Going back to the Popoviciu's Inequality:

$$u^T H u = \mathbb{V}[H] \leq \frac{1}{4} (u_{max} - u_{min})^2 \leq \frac{1}{4} 2\|u\|^2 = \frac{1}{2} \|u\|^2 = \frac{1}{2} u u^T \quad (23)$$

By definition λ_{max} can never exceed $\frac{1}{2}$. Therefore, in terms of the Loewner order:

$$P_i - p_i p_i^\top \preceq \frac{1}{2} I_k \longleftarrow (P_i - p_i p_i^\top) - \left(\frac{1}{2} I_k\right) \text{ is a semi-definite positive matrix} \quad (24)$$

To find the global Lipschitz constant L , let's evaluate the quadratic form of the Hessian for any matrix $V \in \mathbb{R}^{d \times k}$:

$$\text{vec}(V)^\top \nabla^2 f(X) \text{vec}(V) = \sum_{i=1}^m a_i^\top V (P_i - p_i p_i^\top) V^\top a_i \quad (25)$$

Substitute the bound $P_i - p_i p_i^\top \preceq \frac{1}{2} I_k$:

$$\text{vec}(V)^\top \nabla^2 f(X) \text{vec}(V) \leq \frac{1}{2} \sum_{i=1}^m a_i^\top V V^\top a_i = \frac{1}{2} \sum_{i=1}^m \|V^\top a_i\|_2^2 \quad (26)$$

The sum $\sum_{i=1}^m \|V^\top a_i\|_2^2$ is equivalent to $\|AV\|_F^2$, which is bounded by $\lambda_{\max}(A^\top A) \|V\|_F^2$. Therefore, the global Lipschitz constant is:

$$L = \frac{1}{2} \lambda_{\max}(A^\top A) \quad (27)$$

The algorithm for block coordinates gradient descent requires the Lipschitz constant for that specific block, denoted as L_j . By fixing the feature index to j :

$$\nabla_{x_j}^2 f(X) = \sum_{i=1}^m a_{ij}^2 (P_i - p_i p_i^\top) \quad (28)$$

Using the exact same probability bound $(P_i - p_i p_i^\top \preceq \frac{1}{2} I_k)$, we get:

$$\nabla_{x_j}^2 f(X) \preceq \left(\frac{1}{2} \sum_{i=1}^m a_{ij}^2 \right) I_k \quad (29)$$

Therefore, the Lipschitz constant for the j -th block is:

$$L_j = \frac{1}{2} \sum_{i=1}^m a_{ij}^2 \quad (30)$$

This is half the squared $L2$ -norm of the j -th column of the data matrix A .

3 Algorithm

This section describes the optimization algorithms used to solve the multiclass logistic regression problem introduced in the previous section. Although the objective function is convex, the problem becomes computationally demanding when the number of samples m , the dimension d , and the number of classes k are large. For this reason, it is necessary to adopt efficient optimization methods that balance computational cost and convergence speed.

As mentioned before, three different approaches are considered: Gradient Descent, Randomized Block Coordinate Descent, and Block Coordinate Descent with Gauss–Southwell rule. These methods differ in how they exploit the structure of the problem and in the amount of information used at each iteration. In particular, Gradient Descent updates all variables simultaneously using the full gradient, while block coordinate methods update only a subset of variables, leading to cheaper iterations. For all these algorithms, both fixed stepsize and Armijo rule have been implemented as criterion to select the stepsize.

Given the objective function, gradient based methods are a good choice of algorithms. The objective is convex any local minima is also a global minima. Since the algorithms move selecting a descent direction, in this context they can ensure to find a global minima. Obviously, the aim of this analysis is exploiting the algorithms and their properties and the objective given in order to optimize the algorithms; the finding of a minimum is not the final end of the implementation. Still, these theory results demonstrate how gradient based methods are a good choice for the analysis.

3.1 Gradient Descent

Algorithm 1 Gradient Descent

Require: Initial point $X^{(0)}$, objective function $f(X)$, data matrix $A \in \mathbb{R}^{n \times d}$, one-hot labels Y , `armijo_flag`, tolerance ϵ , maximum iterations K , max time T_{max}

Ensure: Optimized weights X , iteration count t , arrays of time, loss, accuracy

```
1: Compute Lipschitz constant stepsize (if avoiding Armijo rule)
2: Initialize logits  $Z \leftarrow AX$ 
3: for  $t = 0$  to  $K - 1$  do
4:   If using Armijo rule, update stepsize  $\alpha$ 
5:   Update weights  $X \leftarrow X - \alpha g$ 
6:   Update logits  $Z \leftarrow AX$ 
7:   Record loss, gradient, accuracy and current timer value
8:   if  $t > 0$  and  $\|g\|_2 < \epsilon$  and current timer  $< T_{max}$  then            $\triangleright$  Stopping condition
9:     break
10:  end if
11: end for
12: return  $X$ ,  $t$ , and history arrays
```

Standard Gradient Descent is an optimization algorithm that exploits the full gradient at each iteration to update the model parameters. While conceptually simple, its computational cost can be significant in large-scale Data Science problems.

In the implementation, the matrix of logits Z is computed and stored at each iteration. This allows to reuse this quantity to efficiently evaluate all the necessary components of the algorithm: the loss function, the gradient and the accuracy. Although this approach may seem simple, it avoids redundant matrix multiplications and reduces the overall computational

burden.

Among the 3 methods considered, Gradient Descent is the most computationally expensive per iteration. At each iteration t , it requires computing a matrix multiplication between the data matrix $A \in \mathbb{R}^{n \times d}$ and the parameter matrix $X \in \mathbb{R}^{d \times k}$, as well as evaluating the full gradient of the objective function. This leads to a per-iteration complexity of $O(ndk)$, where n is the number of samples, d the number of features, and k the number of classes.

Both fixed step sizes based on the Lipschitz constant of the gradient and adaptive step sizes obtained via the Armijo rule were considered. The Armijo rule has a higher per-iteration computational cost, as it requires an inner loop to determine a suitable step size that satisfies a sufficient decrease condition; in particular, at each iteration, the initial step size is multiplied by a given constant $\alpha_{k+1} = \alpha_k * \delta$ and the weight matrix is updated requiring an evaluation of the loss function. The Armijo condition is given by: $f(x_k + \alpha d_k) \leq f(x_k) + \gamma \alpha \nabla f(x_k)^T d_k$, where d_k is a descent direction (in our case $d_k = -\nabla f(x_k)$) and $\gamma \in (0, 1)$ is a predefined parameter. During the experiments, we observed that the Armijo rule results in a lower iteration rate (in terms of iterations per second) compared to the fixed step size based on the Lipschitz constant. On the other hand, it typically achieved convergence in fewer iterations, reflecting its more adaptive step size selection.

Despite its higher per-iteration cost, Gradient Descent typically shows stable and predictable convergence behavior, requiring fewer iterations compared to coordinate-based methods. However, this advantage is offset by its computational cost, making it less efficient in terms of CPU time when dealing with high-dimensional or large-scale datasets.

3.2 Randomized Block Coordinates Gradient Descent

The randomized block coordinates gradient descent selects a block of the weight matrix X for which the gradient is computed, and then the weights are updated. The randomized selects the block casually. In this analysis, a block corresponds to a row of the weight matrix. This method allows for a easier computation of the gradient, because the block can be selected at the start of each iteration casually, without having to compute the whole gradient, this translate into faster updates of the weights, and in general a faster convergence of the algorithm.

Algorithm 2 Randomized Block Coordinate Gradient Descent

Require: Data $A \in \mathbb{R}^{n \times d}$, labels Y , weights X , max iter K , tol ϵ

```

1: Initialize logits  $Z \leftarrow AX$ 
2: Precompute fixed step sizes  $\alpha_j$  (if avoiding line search)
3: for  $t = 0, \dots, K - 1$  do
4:   Sample random block index  $j \sim \mathcal{U}(1, d)$ 
5:   Compute block gradient:  $g_j \leftarrow \frac{1}{n} A_{:,j}^T (\text{softmax}(Z) - Y)$ 
6:   if  $t \bmod d == 0$  and  $\|\frac{1}{n} A^T (\text{softmax}(Z) - Y)\|_2 < \epsilon$  then
7:     break
8:   end if
9:    $\alpha_t \leftarrow \text{Armijo\_function}(g_j, Z, X)$  or fixed  $\alpha_j$ 
10:  Update weights:  $X_{j,:} \leftarrow X_{j,:} - \alpha_t g_j$ 
11:  Update logits:  $Z \leftarrow Z - \alpha_t (A_{:,j} \otimes g_j)$ 
12: end for
13: return  $X$ 
```

At each iteration t , the index j is randomly selected and then the gradient only of the block and its norm are computed. The stop condition is the same of the gradient descent: the norm of the full gradient and the running time. The algorithm stops if the gradient matrix is small

enough or if we reached the maximum time available.

The condition is not the norm of the gradient of the single block because the dataset used is MNIST, that's an image dataset. This dataset is sparse, so to avoiding randomly choosing a block where the gradient is all 0's (a vector of all 0's as a null gradient), the whole gradient is computed. Differently from the gradient descent, the stop condition here is checked each d iteration, this choice was made to avoid computing the whole gradient at each iteration, not increasing the complexity much. This choice makes sense thinking that there isn't a big loss of information by not checking the stop condition each iteration.

In the **for** cycle there is an **if** checking the preferred stepsize approach to use, if fixed or through the Armijo rule. The Armijo function will be analyzed in detail later.

After the computation of the stepsize the update of the weights, the logits are updated too. Here there is another important observation to make: how the logits are updated. Computing again the logits by matrix multiplication would be too expensive and the advantage of block coordinate methods would be lost. This update uses exploit the fact that the algorithm works only on one block. After the **for** all the quantity of interest are returned.

The time complexity of the algorithm is given mostly by the matrix multiplication between data and weights and the computation of the gradient each d iteration. This operation is done once outside the loop and has complexity $O(ndk)$, where n is the number of data, d is the number of features and k is the number of classes. Inside the **for** loop, the most expensive operation is the computation of the logits, with order $O(nk)$. Other operations like the computation of the gradient of the block or the update of Z have the same complexity. The only thing more complex is the computation of the full gradient $O(ndk)$, but since it happens once every d iterations, it reduces to $O(nk)$. It'll be analyzed later, but for now let's assume that the Armijo makes m iterations on average, and each iteration takes $O(nk)$ time. The final complexity of the algorithm is $O(ndk + Knk)$ when a fixed stepsize is used and $O(ndk + Kmnk)$ when the Armijo rule is used; K is the maximum number of iteration, while k is the number of classes.

3.3 Gauss-Southwell Block Coordinates Gradient Descent

Algorithm 3 Randomized Block Coordinate Gradient Descent

Require: Data $A \in \mathbb{R}^{n \times d}$, labels Y , weights X , max iter K , tol ϵ

```

1: Initialize logits  $Z \leftarrow AX$ 
2: Precompute fixed step sizes  $\alpha_j$  (if avoiding line search)
3: for  $t = 0, \dots, K - 1$  do
4:   Compute full gradient:  $g \leftarrow \frac{1}{n} A^T (\text{softmax}(Z) - Y)$ 
5:   Find the block with the maximum norm of the gradient and store its index  $j$ 
6:   if  $t \bmod d == 0$  and  $\|g_j\|_2 < \epsilon$  then
7:     break
8:   end if
9:    $\alpha_t \leftarrow \text{Armijo\_function}(g_j, Z, X)$  or fixed  $\alpha_j$ 
10:  Update weights:  $X_{j,:} \leftarrow X_{j,:} - \alpha_t g_j$ 
11:  Update logits:  $Z \leftarrow Z - \alpha_t (A_{:,j} \otimes g_j)$ 
12: end for
13: return  $X$ 
```

The Gauss-Southwell is pretty similar to the randomized version shown before. There are 2 main differences: the first one is that the norm of the gradient of each block is computed at each iteration now, and not only when we want to check the stop condition. Only after, the index's block is found. The second main difference is the stop condition, since here the

maximum norm of the gradient of the block is computed, there is no need to compute the norm of the whole gradient, that would be a waste of space and time, instead the norm of the gradient of the selected block is taken. Because this norm is the largest, if this quantity is smaller than the tolerance, this means that also all the other blocks have a smaller norm of the gradient. Even if not the same condition, it remains similar and avoid to add complexity to the algorithm.

The complexity analysis is different from the randomized version because each iteration computes the whole gradient, so the complexity increases. Outside the loop, it remains the same, but inside the complexity becomes $O(ndk)$ for the reasons previously given. The overall complexity with fixed stepsize is the following $O(ndk(K + 1))$, while with the Armijo rule is $O(ndk + Knmd)$

3.4 Choice of the Stepsize Rule

There are 3 ways to select the stepsize: fixed value, Armijo rule and exact line search. The last option was not implemented because, given the objective, the computation of the optimal stepsize is not possible. Finding the α minimizing $\phi(\alpha) = f(x_k + \alpha d_k)$ would require a closed form solution that it doesn't exist with this objective. The solution would be a numerical solution, computing the gradient and finding the optimal value. The final algorithm would have a loop inside the main loop, this procedure is too expensive and not feasible.

The other 2 methods have been implemented. The first is the fixed stepsize, the choice of using the inverse of the Lipschitz constant was done in order to hold all the theoretical results about convergence of the algorithms.

The pseudocode of the Armijo is the following: A safety check was inserted to avoid an infinite

Algorithm 4 Armijo Rule function

Require: Loss function, current logits Z , current loss f_x , active data column a_j , block gradient g_j , decay factor δ , parameter γ , initial step size Δ_k

Ensure: Optimal step size α

```

1:  $\alpha \leftarrow \Delta_k$ 
2:  $N_g \leftarrow \|g_j\|_2^2$ 
3: while True do
4:    $Z_{temp} \leftarrow Z - \alpha(a_j \otimes g_j)$ 
5:   if  $loss(Z_{temp}) \leq f_x - \gamma\alpha N_g$  then
6:     break
7:   end if
8:    $\alpha \leftarrow \alpha \cdot \delta$ 
9:   if  $\alpha \leq 10^{-20}$  then
10:    break
11:  end if
12: end while
13: return  $\alpha$ 

```

loop. Outside the loop, the most expensive operation is the squared of the norm of g_j , that has complexity $O(k)$. Inside the loop the complexity is given by the outer product, the update of the logits and the computation of the loss function; all 3 operations have complexity $O(nk)$. Overall, the complexity of the algorithm is $O(k + mnk)$, where m is the number of iterations of the **while** loop.

In the Armijo, there are 3 hyperparameter to choose: Δ_k, δ, γ . For all 3 algorithms, a good rule of thumb for initializing the hyperparameters, selecting the following values: $\Delta_k = 1.0, \gamma =$

$1 * 10^{-4}, \delta = 0.5$. However, the Armijo is an algorithm robust the hyperparameter initialization, so regardless to the starting values of these hyperparameters, it reaches a good values of the stepsize.

4 Results

This section describes the results obtained from testing the algorithms on simulated data and real world data.

4.1 Simulated dataset

The simulated dataset consists of $n = 1000$ samples, $d = 1000$ features, and $k = 50$ classes. The data matrix $A \in \mathbb{R}^{1000 \times 1000}$ is generated with independent standard normal entries, i.e., $A_{ij} \sim \mathcal{N}(0, 1)$. Similarly, the true parameter matrix $X^* \in \mathbb{R}^{1000 \times 50}$ and the noise matrix $E \in \mathbb{R}^{1000 \times 50}$ are generated with independent standard normal variables.

The class labels $Y \in 0, \dots, k - 1^{1000}$ are obtained by applying the softmax function to the matrix $AX^* + E$, and assigning to each sample the index of the maximum probability, i.e., $Y_i = \arg \max_{j \in 0, \dots, k-1} \{\text{softmax}((AX^* + E)_{i,j})\}$.

Starting from an initial parameter matrix X , also randomly initialized with standard Gaussian random variables, all three algorithms are executed on the simulated dataset, with a maximum time limit set at 600 seconds. The corresponding results are reported in Figures 1 and 2.

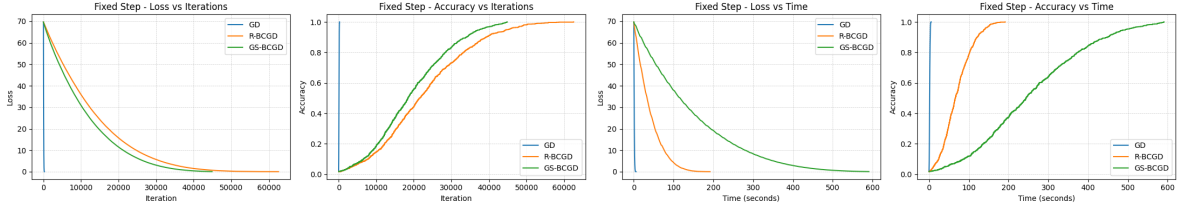


Figure 1: Performance of the algorithms with fixed step size on simulated data.

In the first set of experiments, where all algorithms are run with a fixed step size, we observe that standard Gradient Descent achieves the best performance, reaching perfect accuracy and converging very rapidly (247 iterations in 6 seconds). The Randomized BCGD method also attains perfect accuracy, but requires approximately 62,000 iterations and about 200 seconds (310 iterations per second) to converge. The Gauss–Southwell BCGD method performs fewer iterations (around 45,000), but does not fully converge within the imposed time limit of 10 minutes, although it still achieves near-perfect accuracy.

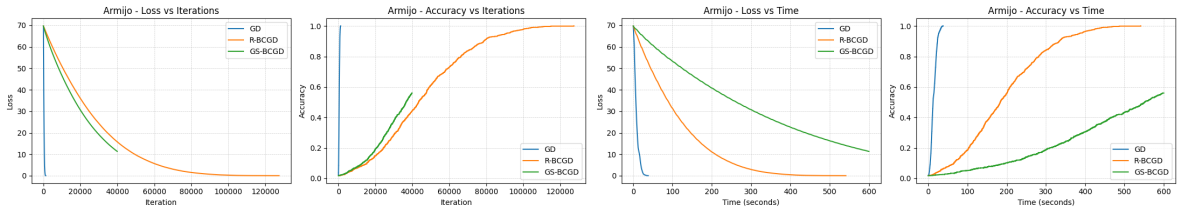


Figure 2: Performance of the algorithms with Armijo step size rule on simulated data.

We repeat the same experiment using the Armijo step size rule and observe that the overall behavior of the algorithms remains largely unchanged. Standard Gradient Descent still achieves the fastest convergence and reaches perfect accuracy. The Randomized BCGD method also converges to 100% accuracy, but requires approximately 125,000 iterations and about 550 seconds (227 iterations per second), reflecting the additional computational cost introduced by the line search procedure. The Gauss–Southwell BCGD method again performs fewer iterations

(around 40,000), but does not converge within the allotted time and attains a final accuracy of approximately 60%.

Thanks to the relatively small dimensionality of the dataset, Gradient Descent outperforms the other algorithms. This can be attributed to the fact that computing the full gradient is not prohibitively expensive in this example, allowing the method to take more informative updates at each iteration. On the other hand, coordinate-based methods update only a subset of variables at a time, which reduces the per-iteration cost but requires a significantly larger number of iterations to reach convergence.

4.2 Real world data

To further evaluate the performance of the algorithms, we conduct experiments on the MNIST dataset, which consists of grayscale 28×28 images of handwritten digits. Each pixel is treated as a feature, resulting in a data matrix $A \in \mathbb{R}^{60000 \times 784}$. The corresponding parameter matrix is $X \in \mathbb{R}^{784 \times 10}$, where each column represents the weights associated with one of the ten digit classes.

On this dataset, we conducted two experiments. In the first case, the algorithms were applied to the raw data without any preprocessing, and so letting each pixel vary in the range $[0, 254]$. In the second case, the data were scaled using a `MinMax` scaler, mapping each feature to the range $[0, 1]$. This preprocessing step was introduced to improve numerical stability and potentially accelerate convergence.

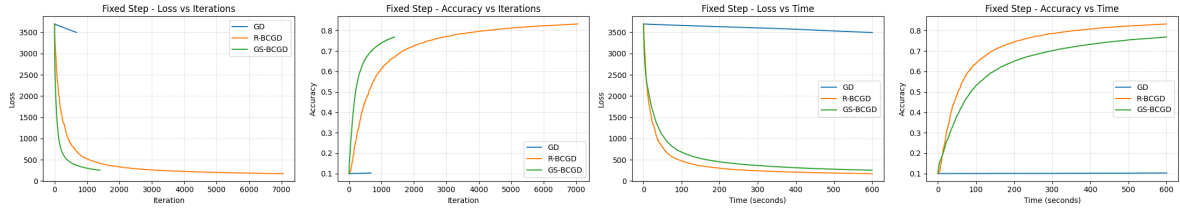


Figure 3: Performance of the algorithms with fixed step size on raw MNIST data.

In this graph, it is clear that the Gradient Descent method is not effective: even when running until the maximum time limit, it performs fewer than 1,000 iterations and achieves roughly 10% accuracy. On the other hand, the BCGD variants perform significantly better than Gradient Descent. Although both run until the time limit, the randomized BCGD completes more than 7,000 iterations and reaches a relatively high accuracy of over 80%. In contrast, the Gauss–Southwell variant performs fewer than 1,500 iterations but achieves almost the same accuracy as the randomized version, suggesting that it selects more informative coordinate updates at each step.

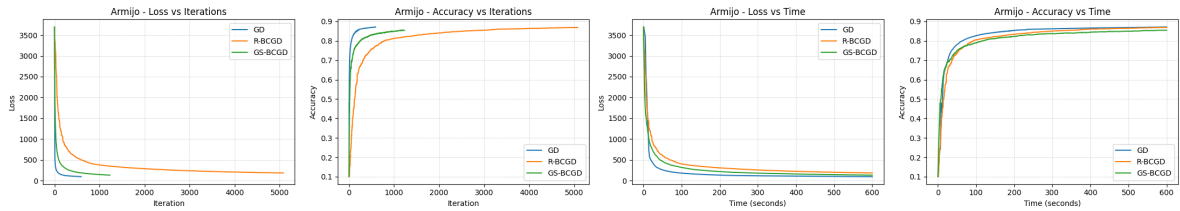


Figure 4: Performance of the algorithms with Armijo step size rule on raw MNIST data.

In this experiment, the application of the Armijo rule significantly improves the performance of Gradient Descent, which reaches nearly 90% accuracy in just over 500 iterations. The Gauss–Southwell variant behaves similarly to the fixed step size case, achieving more than 80%

accuracy in approximately 1,000 iterations. The randomized variant also exhibits behavior comparable to the fixed step size setting, achieving a similar accuracy in fewer iterations; however, this is explained by the additional computational cost introduced by the Armijo rule at each iteration.

Overall, this experiment shows that algorithms using the Armijo rule achieve higher accuracy in fewer iterations compared to the fixed step size approach. However, the loss-versus-time and accuracy-versus-time plots display almost identical trends across methods, suggesting that no algorithm clearly outperforms the others when computational time is taken into account.

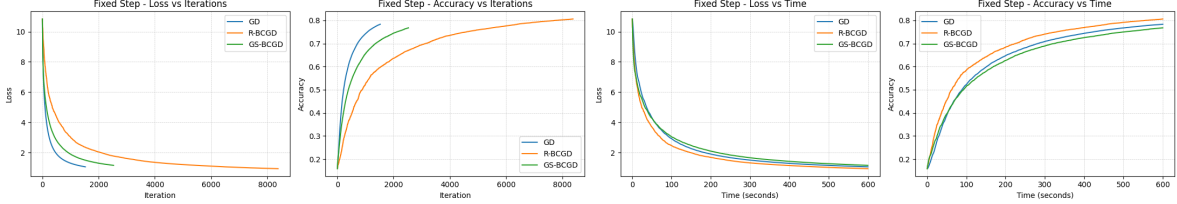


Figure 5: Performance of the algorithms with fixed step size on scaled MNIST data.

On the scaled MNIST dataset, the results with fixed stepsize show the classical trade-off between full-gradient and block-coordinate methods. Gradient Descent reduces the loss very quickly in terms of iterations, since each update uses the full gradient and is therefore more informative. However, because each iteration is computationally expensive, its advantage is less pronounced when performance is evaluated over CPU time. Randomized BCGD converges more slowly in terms of iterations, but its much cheaper updates make it competitive in time, and it even achieves slightly better accuracy in the long run. Gauss–Southwell BCGD behaves similarly to Gradient Descent in terms of convergence pattern, since selecting the block with the largest gradient norm makes each update more effective than a random choice, although this comes at a higher computational cost.

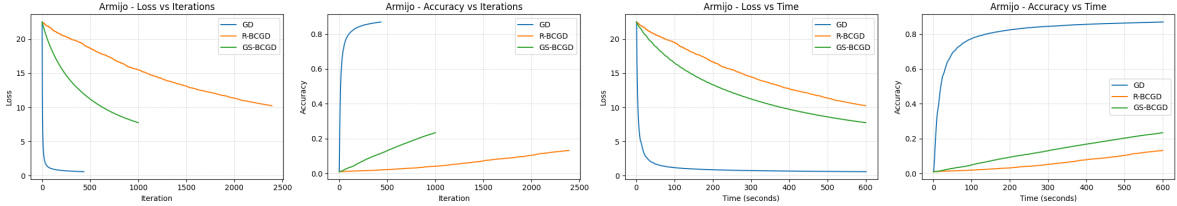


Figure 6: Performance of the algorithms with Armijo step size rule on scaled MNIST data.

When the Armijo rule is used, the qualitative behavior of the methods changes significantly. Gradient Descent still performs best, as the adaptive stepsize allows a rapid decrease of the loss while preserving stability. In contrast, the coordinate-based methods are heavily penalized by the line-search procedure: although Armijo improves the reliability of the updates, the additional cost of repeatedly evaluating the objective function becomes dominant. As a consequence, both Randomized and Gauss–Southwell BCGD exhibit much slower progress in both loss and accuracy. This confirms that, on large-scale real data, adaptive stepsize rules may improve robustness, but they can also introduce a substantial computational overhead, especially for coordinate methods.

5 LLM Usage Disclosure Statement

In this section, we explain how and the reasons that drove us to use LLMs. Gemini 3 and ChatGPT were used as support tools for three main purposes: debugging the code, improving code readability, and obtaining guidance for the visualization of the results. A relevant use of LLMs concerned the plotting section of the project. Since clear and consistent plots were needed to properly represent the behavior of the methods. As there was limited prior experience with plotting in python, LLMs were used to obtain suggestions on how to structure the figures and make the visual comparison more understandable. LLMs were also used to help identify and fix coding errors, as well as to make the implementation cleaner and more readable. In addition, they were used as a support tool to better understand some theoretical derivations. All final algorithmic choices, mathematical derivations, code implementations, and interpretations of the results were reviewed and validated by us. Therefore, LLMs were used only as auxiliary tools for debugging, visualization support, and theoretical clarification.